

Лабораторная работа №5

Аппарат сетей Петри: задача обедающих философов

Назармамадов Умед Джамshedович

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение лабораторной работы	8
4.1	Подготовка окружения	8
4.2	Реализация модели	9
4.3	Базовые эксперименты	10
4.4	Анимация процесса	11
4.5	Итоговый сравнительный отчёт	12
4.6	Генерация производных форматов	13
5	Выводы	14
	Список литературы	15

Список иллюстраций

4.1	Создание структуры проекта и запуск Julia	8
4.2	Инициализация проекта DrWatson	8
4.3	Установка необходимых пакетов	9
4.4	Код модуля src/DiningPhilosophers.jl	10
4.5	Код скрипта dining_philosophers.jl	10
4.6	Запуск базовых экспериментов	11
4.7	Код скрипта dining_philosophers_animation.jl	11
4.8	Запуск генерации анимации	12
4.9	Код скрипта dining_philosophers_report.jl	12
4.10	Код скрипта tangle.jl	13
4.11	Выполнение Jupyter Notebook	13

Список таблиц

1 Цель работы

Целью данной лабораторной работы является изучение аппарата сетей Петри и его применение для моделирования классической задачи синхронизации «Обедающие философы», включая исследование проблемы взаимной блокировки (deadlock) и способов её предотвращения.

2 Задание

1. Создать рабочий каталог для кода и установить необходимые пакеты.
2. Реализовать модуль сетей Петри с построением классической сети (без арбитра) и сети с арбитром.
3. Провести стохастическое моделирование (алгоритм Гиллеспи) для обеих сетей и определить наличие deadlock.
4. Построить анимацию динамики маркировки сети.
5. Построить сводный сравнительный график состояний «Ест» для обеих моделей.
6. Преобразовать код в литературный стиль и сгенерировать чистый код, документацию Quarto и Jupyter Notebook.
7. Выполнить код из Jupyter Notebook.

3 Теоретическое введение

Сеть Петри — математический аппарат для моделирования дискретных систем, представляемый как двудольный ориентированный граф с двумя типами вершин: позиции (места) и переходы. Дуги соединяют позиции с переходами и наоборот, а маркировка (распределение фишек по позициям) описывает текущее состояние системы. Переход считается разрешённым, если каждая его входная позиция содержит достаточное для срабатывания количество фишек; срабатывание перехода удаляет фишки из входных позиций и добавляет их в выходные.

Задача «Обедающие философы», сформулированная Эдсгером Дейкстрой в 1965 году, иллюстрирует проблему взаимной блокировки (deadlock) при конкурентном доступе к разделяемым ресурсам. N философов сидят за круглым столом, между каждой парой соседей лежит одна вилка, и философу для еды нужны обе соседние вилки. При наивной реализации, когда каждый философ сначала берёт левую вилку, а затем правую, возможна ситуация, когда все философы одновременно возьмут левую вилку и будут бесконечно ждать правую — это и есть deadlock. Один из способов предотвращения тупика — введение арбитра, ограничивающего число философов, одновременно претендующих на вилки, до $N-1$.

4 Выполнение лабораторной работы

4.1 Подготовка окружения

Для реализации модели был установлен язык Julia и необходимые пакеты: OrdinaryDiffEq, DataFrames, CSV, Plots и DrWatson.

```
umedn@HUAWEI:~/work/study/2026-1/2026-1-study-simulation-modeling$ mkdir -p labs/lab05/project/script
umedn@HUAWEI:~/work/study/2026-1/2026-1-study-simulation-modeling$ mkdir -p labs/lab05/project/src
umedn@HUAWEI:~/work/study/2026-1/2026-1-study-simulation-modeling$ cd labs/lab05
umedn@HUAWEI:~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab05$ julia

Documentation: https://docs.julialang.org
Type "?" for help, "]"? for Pkg help.
Version 1.10.5 (2024-08-27)
Official https://julialang.org/ release

julia> |
```

Рисунок 4.1: Создание структуры проекта и запуск Julia

```
julia> using DrWatson

julia> initialize_project("project"; authors="Umed Nazarmamadov", git=false)
Activating new project at `~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab05/project`
Resolving package versions...
Updating `~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab05/project/Project.toml`
[634d3b9d] + DrWatson v2.19.1
Updating `~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab05/project/Manifest.toml`
[0b6fb165] + ChunkCodeCore v1.0.2
[4c0bbe4] + ChunkCodeLibZlib v1.1.0
[55437552] + ChunkCodeLibZstd v1.0.0
[634d3b9d] + DrWatson v2.19.1
[5789e2e9] + FileIO v1.20.0
[076d061b] + HashArrayMappedTries v0.2.0
[033835bb] + JLD2 v0.6.6
[692b3bcd] + JLLWrappers v1.8.0
[1914dd2f] + MacroTools v0.5.16
[bac558e1] + OrderedCollections v2.0.1
[aea7be01] + PrecompileTools v1.2.1
```

Рисунок 4.2: Инициализация проекта DrWatson


```

umedn@HUAWEI:~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab05/project$ julia --project
=. -e "using Pkg; Pkg.add([\"DrWatson\", \"DataFrames\", \"Plots\", \"CSV\", \"OrdinaryDiffEq\", \"Ra
ndom\", \"LinearAlgebra\"])"
Resolving package versions...
Updating `~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab05/project/Project.toml`
[336ed68f] + CSV v0.10.17
[a93c6f00] + DataFrames v1.8.2
[1dea7af3] + OrdinaryDiffEq v7.8.1
[91a5bcd] + Plots v1.41.7
[37e2e46d] + LinearAlgebra
[9a3f8284] + Random
Updating `~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab05/project/Manifest.toml`
[47edcb42] + ADTypes v1.24.0
[14f7f29c] + AMD v0.5.3
[7d9f7c33] + Accessors v0.1.45
[79e6a3ab] + Adapt v4.7.0
[66dad0bd] + AliasTables v1.1.3
[4fba245c] + ArrayInterface v7.30.1
[b2a6c25c] + BinaryHeaps v1.1.0
[70df07ce] + BracketingNonlinearSolve v1.12.6
[336ed68f] + CSV v0.10.17
[944b1d66] + CodecZlib v0.7.9
[35d6a980] + ColorSchemes v3.31.0
[3da002f7] + ColorTypes v0.12.1
[c3611d14] + ColorVectorSpace v0.11.0
[5ae59095] + Colors v0.13.1

```

Рисунок 4.3: Установка необходимых пакетов

4.2 Реализация модели

Модель сети Петри реализована в модуле `src/DiningPhilosophers.jl`. Определена структура `PetriNet` с матрицей инцидентности, задающей входные и выходные дуги переходов. Функция `build_classical_network` строит классическую сеть для N философов без синхронизации, а `build_arbiter_network` добавляет дополнительную позицию-арбитр с $N-1$ фишками, ограничивающую число философов, одновременно берущих вилки. Реализованы детерминированное моделирование на основе ОДУ и стохастическое моделирование прямым методом Гиллеспи, а также функция `detect_deadlock` для проверки, остались ли в системе разрешённые переходы.

```

umedn@HUAWEI: ~/work/stu x + v
...study/2026-1/2026-1-study-simulation-modeling/labs/lab05/project/src/DiningPhilosophers.jl *
module DiningPhilosophers

using OrdinaryDiffEq
using Plots
using DataFrames
using Random, LinearAlgebra

export build_classical_network, build_arbiter_network
export simulate_ode, simulate_stochastic
export detect_deadlock, plot_marking_evolution

struct PetriNet
    n_places::Int
    n_transitions::Int
    incidence::Matrix{Int}
    place_names::Vector{Symbol}
    transition_names::Vector{Symbol}
end

function PetriNet(
    n_places,
    n_transitions;
    place_names = Symbol[],
    transition_names = Symbol[],

```

Рисунок 4.4: Код модуля src/DiningPhilosophers.jl

4.3 Базовые эксперименты

Для сравнения классической сети и сети с арбитром создан файл scripts/dining_philosophers.jl. Для N=5 философов проведена стохастическая симуляция на время tmax=50 для обеих сетей, результаты сохранены в CSV и проверены на наличие deadlock.

```

umedn@HUAWEI: ~/work/stu x + v
.../2026-1/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining_philosophers.jl *
using DrWatson
@quickactivate "project"
include(sourcedir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using DataFrames, CSV, Plots

N = 5
tmax = 50.0

println("=== Классическая сеть (без арбитра) ===")
net_classic, u0_classic, _ = build_classical_network(N)
df_classic = simulate_stochastic(net_classic, u0_classic, tmax)

mkpath(datadir())
mkpath(plotsdir())

CSV.write(datadir("dining_classic.csv"), df_classic)
dead = detect_deadlock(df_classic, net_classic)
println("Deadlock обнаружен: $dead")

plot_classic = plot_marking_evolution(df_classic, N)
savefig(plotsdir("classic_simulation.png"))

println("\n=== Сеть с арбитром ===")

```

Рисунок 4.5: Код скрипта dining_philosophers.jl

```

umedn@HUAWEI:~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab05$ julia --project=project
project/scripts/dining_philosophers.jl
=== Классическая сеть (без арбитра) ===
Deadlock обнаружен: true

=== Сеть с арбитром ===
Deadlock обнаружен: false
Готово!
umedn@HUAWEI:~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab05$ |

```

Рисунок 4.6: Запуск базовых экспериментов

Результаты подтвердили теоретические предсказания: в классической сети без синхронизации возникает взаимная блокировка — все философы переходят в состояние «Голоден» и остаются в нём навсегда, в то время как сеть с арбитром демонстрирует постоянное чередование приёма пищи без блокировок.

4.4 Анимация процесса

Для наглядной

ии динамики работы сети Петри во времени создан файл `scripts/dining_philosophers_animation.jl`, генерирующий GIF-анимацию изменения маркировки для классической сети с $N=3$ философами.

```

...26-1-study-simulation-modeling/labs/lab05/project/scripts/dining_philosophers_animation.jl *
using DrWatson
@quickactivate "project"
include(sreaddir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using Plots, Random

N = 3
tmax = 30.0

net, u0, names = build_classical_network(N)
Random.seed!(123)
df = simulate_stochastic(net, u0, tmax)

anim = @animate for row in eachrow(df)
    u = [row[col] for col in propertynames(row) if col != :time]
    bar(
        1:length(u),
        u,
        legend = false,
        ylims = (0, maximum(u0) + 1),
        xlabel = "Позиция",
        ylabel = "Фишки",
        title = "Время = $(round(row.time, digits=2))",
    )
end

```

Рисунок 4.7: Код скрипта `dining_philosophers_animation.jl`

```

umedn@HUAWEI:~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab05$ julia --project=project
project/scripts/dining_philosophers_animation.jl
[ Info: Saved animation to /home/umedn/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab05/
project/plots/philosophers_simulation.gif
Анимация сохранена в plots/philosophers_simulation.gif
umedn@HUAWEI:~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab05$ |

```

Рисунок 4.8: Запуск генерации анимации

Анимация наглядно показывает перемещение фишек между позициями «Думает», «Голоден», «Ест» и «Вилка», а также застывание маркировки на последних кадрах, соответствующее наступлению deadlock.

4.5 Итоговый сравнительный отчёт

Для количественного сравнения двух моделей создан файл `scripts/dining_philosophers_report.jl`, загружающий ранее сохранённые CSV-файлы и строящий сводный график динамики состояний «Ест» для каждого философа в обеих сетях.

```

umedn@HUAWEI: ~/work/stu  ×  +  ▾
.../2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining_philosophers_report.jl *
using DrWatson
@quickactivate "project"
using DataFrames, CSV, Plots

df_classic = CSV.read(datadir("dining_classic.csv"), DataFrame)
df_arbiter = CSV.read(datadir("dining_arbiter.csv"), DataFrame)

N = 5
eat_cols = [Symbol("Eat_$i") for i = 1:N]

p1 = plot(
    df_classic.time,
    Matrix(df_classic[:, eat_cols]),
    label = ["Φ $i" for i = 1:N],
    xlabel = "Время",
    ylabel = "Ест (1/0)",
    title = "Классическая сеть",
)

p2 = plot(
    df_arbiter.time,
    Matrix(df_arbiter[:, eat_cols]),
    label = ["Φ $i" for i = 1:N],
    xlabel = "Время",
)

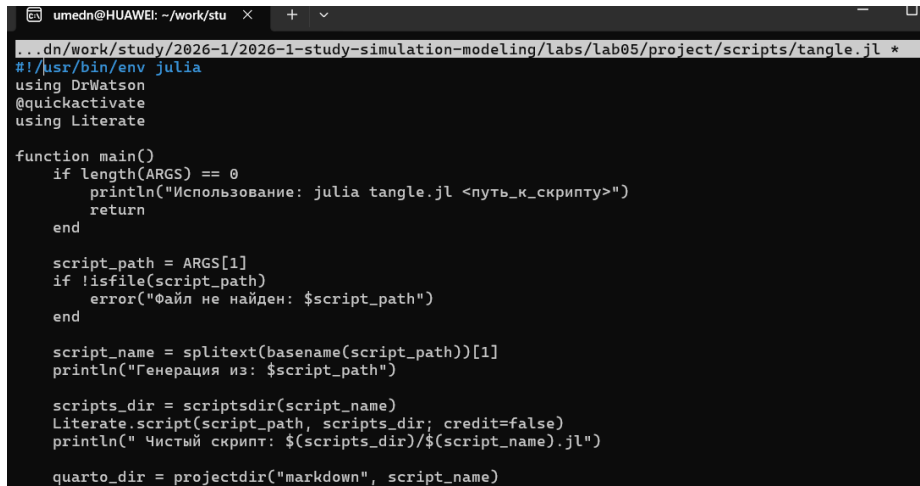
```

Рисунок 4.9: Код скрипта `dining_philosophers_report.jl`

На графике для классической сети все линии состояния «Ест» со временем падают до нуля и остаются на нём — деятельность философов полностью прекращается. Для сети с арбитром линии продолжают колебаться на протяжении всего времени моделирования, что подтверждает эффективность арбитра как механизма предотвращения тупиков.

4.6 Генерация производных форматов

Для преобразования всех трёх скриптов в литературный стиль использован вспомогательный скрипт `tangle.jl` на основе библиотеки `Literate.jl`. Для каждого скрипта сгенерированы чистый исполняемый код, документация в формате Quarto и Jupyter Notebook.



```
umedn@HUAWEI: ~/work/stu x + v
...dn/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab05/project/scripts/tangle.jl *
#!/usr/bin/env julia
using DrWatson
@quickactivate
using Literate

function main()
    if length(ARGS) == 0
        println("Использование: julia tangle.jl <путь_к_скрипту>")
        return
    end

    script_path = ARGS[1]
    if !isfile(script_path)
        error("Файл не найден: $script_path")
    end

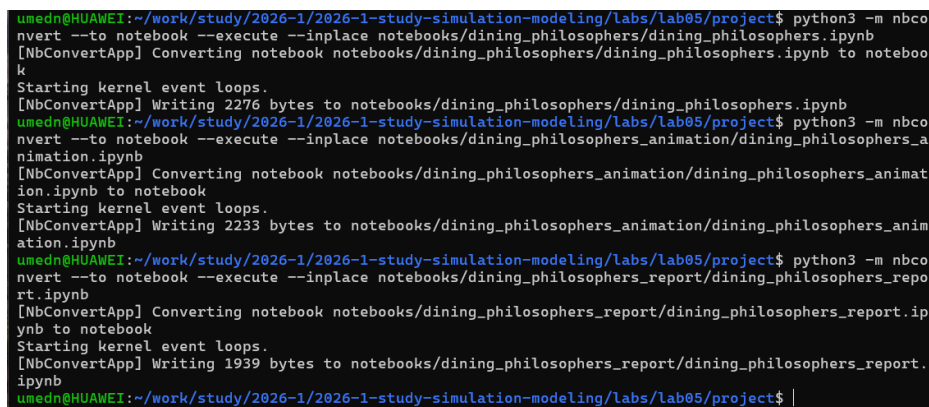
    script_name = splitext(basename(script_path))[1]
    println("Генерация из: $script_path")

    scripts_dir = scriptsdir(script_name)
    Literate.script(script_path, scripts_dir; credit=false)
    println(" Чистый скрипт: $(scripts_dir)/$(script_name).jl")

    quarto_dir = projectdir("markdown", script_name)
end
```

Рисунок 4.10: Код скрипта `tangle.jl`

Все сгенерированные Jupyter Notebook были выполнены для проверки корректности литературного представления кода.



```
umedn@HUAWEI:~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab05/project$ python3 -m nbconvert --to notebook --execute --inplace notebooks/dining_philosophers/dining_philosophers.ipynb
[NbConvertApp] Converting notebook notebooks/dining_philosophers/dining_philosophers.ipynb to notebook
Starting kernel event loops.
[NbConvertApp] Writing 2276 bytes to notebooks/dining_philosophers/dining_philosophers.ipynb
umedn@HUAWEI:~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab05/project$ python3 -m nbconvert --to notebook --execute --inplace notebooks/dining_philosophers_animation/dining_philosophers_animation.ipynb
[NbConvertApp] Converting notebook notebooks/dining_philosophers_animation/dining_philosophers_animation.ipynb to notebook
Starting kernel event loops.
[NbConvertApp] Writing 2233 bytes to notebooks/dining_philosophers_animation/dining_philosophers_animation.ipynb
umedn@HUAWEI:~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab05/project$ python3 -m nbconvert --to notebook --execute --inplace notebooks/dining_philosophers_report/dining_philosophers_report.ipynb
[NbConvertApp] Converting notebook notebooks/dining_philosophers_report/dining_philosophers_report.ipynb to notebook
Starting kernel event loops.
[NbConvertApp] Writing 1939 bytes to notebooks/dining_philosophers_report/dining_philosophers_report.ipynb
umedn@HUAWEI:~/work/study/2026-1/2026-1-study-simulation-modeling/labs/lab05/project$ |
```

Рисунок 4.11: Выполнение Jupyter Notebook

5 Выводы

В ходе лабораторной работы изучен аппарат сетей Петри и реализована классическая задача синхронизации «Обедающие философы». Проведённое стохастическое моделирование подтвердило теоретический вывод о неизбежности взаимной блокировки при наивной реализации без синхронизации, в то время как введение арбитра, ограничивающего число одновременно голодных философов, полностью предотвращает deadlock. Построенные анимация и сводный сравнительный график наглядно продемонстрировали различие в поведении обеих моделей. Весь код преобразован в литературный стиль с генерацией нескольких производных форматов.

Список литературы